Minimum number of swaps

Difficulty: **Easy** Accuracy: **42.47%** Submissions: **688+** Points: **2**

You are given two binary strings **s1** and **s2** of equal length, and the task is to find the **minimum number** of swaps required to make them identical. The only allowed operation is swapping characters between the two strings (i.e., you can swap **s1[i]** with **s2[j]**, but not within the same string). If it is impossible to make the two strings equal through such swaps, return **-1**.

Examples:

Input: s1 = "1100", s2 = "1111"
Output: 1
Explanation: Swap the character at index 2 of s1 with the character at index 3 of s2. After this swap both strings become "1110", so only 1 swap is needed.

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed	Attempts : Correct / Total
1120 / 1120	**1 / 1**
	Accuracy : 100%

Points Scored Time Taken

Java (21) Start Timer

```java
class Solution {
    public int minSwaps(String s1, String s2) {
        int xy = 0;
        int yx = 0;

        for (int i = 0; i < s1.length(); i++) {
            char c1 = s1.charAt(i);
            char c2 = s2.charAt(i);

            if (c1 != c2) {
                if (c1 == '0') {
                    xy++;
                } else {
                    yx++;
                }
            }
        }

        if ((xy + yx) % 2 != 0) {
            return -1;
        }

        int totalSwaps = (xy / 2) + (yx / 2);

        if (xy % 2 != 0) {
            totalSwaps += 2;
        }

        return totalSwaps;
    }
}
```

Custom Input Compile & Run Submit

Min Add to Make Parentheses Valid

Difficulty: **Medium** Accuracy: **57.97%** Submissions: **17K+** Points: **4**

You are given a string **s** consisting only of the characters **'('** and **')'**. Your task is to determine the **minimum** number of parentheses (either '(' or ')') that must be inserted at any positions to make the string s a **valid parentheses string**.

A parentheses string is considered **valid** if:

1. Every opening parenthesis '(' has a corresponding closing parenthesis ')'.
2. Every closing parenthesis ')' has a corresponding opening parenthesis '('.
3. Parentheses are properly nested.

Examples:

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed

1111 / 1111

Attempts : Correct / Total

1 / 1

Accuracy : 100%

Points Scored

Time Taken

Java (21) Start Timer

```java
class Solution {
    public int minParentheses(String s) {
        int open = 0;
        int add = 0;

        for (int i = 0; i < s.length(); i++) {
            if (s.charAt(i) == '(') {
                open++;
            } else {
                if (open > 0) {
                    open--;
                } else {
                    add++;
                }
            }
        }

        return open + add;
    }
}
```

Custom Input Compile & Run Submit

Score of Parentheses String

Difficulty: **Medium** Accuracy: **85.49%** Submissions: **492+** Points: **4**

Given a string **s** consisting of balanced parentheses, calculate the **score** of the string based on the following rules:

- "()" has a score of 1.
- "AB" has a score of A + B, where A and B are balanced parentheses strings.
- "(A)" has a score of 2 × score(A), where A is a balanced parentheses string.

Note: Test cases are generated such that the score will fit within a 32-bit integer.

Examples:

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed	Attempts : Correct / Total
1120 / 1120	**1 / 1** Accuracy : 100%
Points Scored	Time Taken

Java (21) Start Timer

```java
class Solution {
    public int scoreOfParentheses(String s) {
        int score = 0;
        int depth = 0;

        for (int i = 0; i < s.length(); i++) {
            if (s.charAt(i) == '(') {
                depth++;
            } else {
                depth--;
                if (s.charAt(i - 1) == '(') {
                    score += 1 << depth;
                }
            }
        }

        return score;
    }
}
```

Custom Input Compile & Run Submit

Difference Check

Difficulty: **Medium** Accuracy: **56.74%** Submissions: **16K+** Points: **4**

Given an array **arr[]** of time strings in 24-hour clock format **"HH:MM:SS"**, return the **minimum difference** in **seconds** between any two time strings in the arr[].
The clock wraps around at midnight, so the time difference between "23:59:59" and "00:00:00" is **1** second.

Examples:

Input: arr[] = ["12:30:15", "12:30:45"]
Output: 30
Explanation: The minimum time difference is 30 seconds.

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed	Attempts : Correct / Total
1120 / 1120	**1 / 1**
	Accuracy : 100%

Points Scored Time Taken

Java (21) Start Timer

```java
import java.util.Arrays;

class Solution {
    public int minDifference(String[] arr) {
        int n = arr.length;
        int[] seconds = new int[n];

        for (int i = 0; i < n; i++) {
            String[] parts = arr[i].split(":");
            int h = Integer.parseInt(parts[0]);
            int m = Integer.parseInt(parts[1]);
            int s = Integer.parseInt(parts[2]);
            seconds[i] = h * 3600 + m * 60 + s;
        }

        Arrays.sort(seconds);

        int totalSecondsInDay = 86400;
        int minDiff = totalSecondsInDay - (seconds[n - 1] - seconds[0]);

        for (int i = 1; i < n; i++) {
            minDiff = Math.min(minDiff, seconds[i] - seconds[i - 1]);
            if (minDiff == 0) return 0;
        }

        return minDiff;
    }
}
```

Custom Input Compile & Run Submit

Sort by Frequency

Difficulty: **Medium** Accuracy: **67.84%** Submissions: **3K+** Points: **4**

Given a string **s**, the task is to **arrange** the string according to the **frequency** of each character, in ascending order. If two elements have the **same** frequency, then they are sorted in **lexicographical** order.

Examples:

Input: s = "geeksforgeeks"
Output: forggkksseeee
Explanation: All the characters with minimum frequency will occur first and the one with same frequency will be arranged lexicographically.

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed	Attempts : Correct / Total
1117 / 1117	**1 / 1**
	Accuracy : 100%

Points Scored Time Taken

Java (21) Start Timer

```java
class Solution {
    public String frequencySort(String s) {
        int[] freqMap = new int[26];
        for (char ch : s.toCharArray()) {
            freqMap[ch - 'a']++;
        }

        List<Character> chars = new ArrayList<>();
        for (int i = 0; i < 26; i++) {
            if (freqMap[i] > 0) {
                chars.add((char) (i + 'a'));
            }
        }

        Collections.sort(chars, (a, b) -> {
            if (freqMap[a - 'a'] != freqMap[b - 'a']) {
                return freqMap[a - 'a'] - freqMap[b - 'a'];
            }
            return a - b;
        });

        StringBuilder res = new StringBuilder();
        for (char ch : chars) {
            int count = freqMap[ch - 'a'];
            for (int i = 0; i < count; i++) {
                res.append(ch);
            }
        }

        return res.toString();
    }
}
```

Custom Input Compile & Run Submit

Sort an array of strings according to string lengths

Difficulty: **Medium** Accuracy: **49.82%** Submissions: **2K+** Points: **4** Average Time: **10m**

You are given an array **arr[]** of strings. Your have to **sort** the array in **ascending** order based on the **lengths** of the strings. If two strings have the same length, maintain their original **relative order**.

Examples:

Input: arr[] = ["GeeksforGeeeks", "I", "from", "am"]
Output: ["I", "am", "from", "GeeksforGeeks"]
Explanation: The strings are sorted in increasing order of their lengths, starting from the shortest string "I" to the longest one "GeeksforGeeeks".

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed	Attempts : Correct / Total
1111 / 1111	**1 / 1** Accuracy : 100%

Points Scored Time Taken

Java (21) Start Timer

```java
class Solution {
    public void sortByLength(String[] arr) {
        java.util.Arrays.sort(arr, (a, b) -> Integer.compare(a.length(), b.length()));
    }
}
```

Custom Input Compile & Run Submit

Count pairs of strings with one mismatch

Difficulty: **Medium** Accuracy: **69.36%** Submissions: **751+** Points: **4**

Given an array **arr[]** of strings of **equal length**, where each string consists of lowercase English letters, find the **total** number of **pairs of strings** that differ in exactly **one character position**.
Note: Two strings differ in exactly one position if they have the same length and differ in exactly one index that is, all characters except one are the same.

Examples :

Input: arr[] = ["abc", "abd", "bbd"]
Output: 2
Explanation: The valid pairs are:
1. ("abc", "abd") → differ at index 2

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed
1120 / 1120

Attempts : Correct / Total
1 / 1
Accuracy : 100%

Points Scored

Time Taken

Java (21) Start Timer

```java
class Solution {
    public int countPairs(String[] arr) {
        int n = arr.length;
        if (n == 0) return 0;
        int m = arr[0].length();
        int totalPairs = 0;

        for (int j = 0; j < m; j++) {
            java.util.Map<String, Integer> map = new java.util.HashMap<>();
            for (int i = 0; i < n; i++) {
                StringBuilder sb = new StringBuilder(arr[i]);
                sb.setCharAt(j, '*');
                String pattern = sb.toString();

                int count = map.getOrDefault(pattern, 0);
                totalPairs += count;
                map.put(pattern, count + 1);
            }
        }

        java.util.Map<String, Integer> duplicates = new java.util.HashMap<>();
        for (String s : arr) {
            duplicates.put(s, duplicates.getOrDefault(s, 0) + 1);
        }

        int exactMatches = 0;
        for (int count : duplicates.values()) {
            if (count > 1) {
                exactMatches += (count * (count - 1)) / 2;
            }
        }

        return totalPairs - (exactMatches * m);
    }
}
```

Custom Input Compile & Run Submit

Winner of an election

Difficulty: **Easy** Accuracy: **71.01%** Submissions: **775+** Points: **2**

Given a lowercase string array **arr**[]. Each element in the array represents a vote cast for a candidate. Return the name of the candidate who received the maximum number of votes and the count of votes he received. In case of a tie between two or more candidates, return the **lexicographically** smallest name.

Note: Return an array of strings, the winning candidate name as the first element and the vote count as the second element (typecast the count to string).

Examples :

Input: arr[] = ["john", "johnny", "jackie", "johnny", "john", "jackie", "jamie", "jamie",

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed	Attempts : Correct / Total
1110 / 1110	**1 / 1**
	Accuracy : 100%

Points Scored

Time Taken

Java (21) Start Timer

```java
class Solution {
    public List<String> winner(List<String> arr) {
        Map<String, Integer> map = new HashMap<>();
        for (String s : arr) {
            map.put(s, map.getOrDefault(s, 0) + 1);
        }

        String winner = "";
        int maxVotes = 0;

        for (Map.Entry<String, Integer> entry : map.entrySet()) {
            String candidate = entry.getKey();
            int votes = entry.getValue();

            if (votes > maxVotes) {
                maxVotes = votes;
                winner = candidate;
            } else if (votes == maxVotes) {
                if (candidate.compareTo(winner) < 0) {
                    winner = candidate;
                }
            }
        }

        List<String> result = new ArrayList<>();
        result.add(winner);
        result.add(String.valueOf(maxVotes));
        return result;
    }
}
```

Custom Input Compile & Run Submit

Check if Permutation is Substring

Difficulty: **Easy** Accuracy: **38.63%** Submissions: **3K+** Points: **2**

Given two strings **txt** and **pat** having lowercase letters, the task is to check if any permutation of **pat** is a substring of **txt**.

Examples:

Input: txt = "geeks", pat = "eke"
Output: true
Explanation: "eek" is a permutation of "eke" which exists in "geeks".

Input: txt = "programming", pat = "rain"
Output: false

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed
1115 / 1115

Attempts : Correct / Total
1 / 1
Accuracy : 100%

Points Scored

Time Taken

Java (21) Start Timer

```java
class Solution {
    boolean search(String txt, String pat) {
        int n = txt.length();
        int m = pat.length();

        if (m > n) return false;

        int[] patCount = new int[26];
        int[] txtCount = new int[26];

        for (int i = 0; i < m; i++) {
            patCount[pat.charAt(i) - 'a']++;
            txtCount[txt.charAt(i) - 'a']++;
        }

        for (int i = 0; i <= n - m; i++) {
            if (isMatch(patCount, txtCount)) return true;

            if (i < n - m) {
                txtCount[txt.charAt(i) - 'a']--;
                txtCount[txt.charAt(i + m) - 'a']++;
            }
        }

        return false;
    }

    private boolean isMatch(int[] a, int[] b) {
        for (int i = 0; i < 26; i++) {
            if (a[i] != b[i]) return false;
        }
        return true;
    }
}
```

Custom Input Compile & Run Submit

Lexicographically Largest String After K Deletions

Difficulty: **Medium** Accuracy: **53.68%** Submissions: **17K+** Points: **4** Average Time: **15m**

Given a string **s** consisting of lowercase English letters and an integer **k**, your task is to remove exactly k characters from the string. The resulting string must be the largest possible in **lexicographical order**, while maintain the relative order of the remaining characters.

Examples:

Input: s = "ritz", k = 2
Output: tz
Explaination: By removing two characters in all possible ways, we get: "ri", "rt", "rz", "it", "iz", and "tz". Among these, "tz" is lexicographically largest string.

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed
1120 / 1120

Attempts : Correct / Total
1 / 1
Accuracy : 100%

Points Scored

Time Taken

Java (21) Start Timer

```java
class Solution {
    public static String maxSubseq(String s, int k) {
        StringBuilder stack = new StringBuilder();
        int n = s.length();
        int toRemove = k;

        for (int i = 0; i < n; i++) {
            char curr = s.charAt(i);
            while (toRemove > 0 && stack.length() > 0 && stack.charAt(stack.length() - 1) < c
                stack.deleteCharAt(stack.length() - 1);
                toRemove--;
            }
            stack.append(curr);
        }

        return stack.substring(0, n - k);
    }
}
```

Custom Input Compile & Run Submit

Count Unique Vowel Strings

Difficulty: **Medium** Accuracy: **50.99%** Submissions: **20K+** Points: **4**

You are given a lowercase string **s**, determine the total number of distinct strings that can be formed using the following rules:

- Identify all **unique** vowels (a, e, i, o, u) present in the string.
- For each distinct vowel, choose **exactly one** of its occurrences from s. If a vowel appears multiple times, each occurrence represents a unique selection choice.
- Generate all possible permutations of the selected vowels. Each unique arrangement counts as a distinct string.

Return the total number of such distinct strings.

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed	Attempts : Correct / Total
102 / 102	**1 / 1**
	Accuracy : 100%

Points Scored Time Taken

Java (21) Start Timer

```java
class Solution {
    public int vowelCount(String s) {
        int[] freq = new int[256];
        for (char c : s.toCharArray()) {
            freq[c]++;
        }

        char[] vowels = {'a', 'e', 'i', 'o', 'u'};
        long product = 1;
        int distinctCount = 0;

        for (char v : vowels) {
            if (freq[v] > 0) {
                product *= freq[v];
                distinctCount++;
            }
        }

        if (distinctCount == 0) return 0;

        long factorial = 1;
        for (int i = 1; i <= distinctCount; i++) {
            factorial *= i;
        }

        return (int) (product * factorial);
    }
}
```

Custom Input Compile & Run Submit

Count Even Letters

Difficulty: **Easy** Accuracy: **87.05%** Submissions: **2K+** Points: **2**

You are given a string **s** consisting of lowercase english letters. Your task is to count how many **distinct** characters appear an **even** number of times in the string.

Examples:

Input: s = "abacaba"
Output: 2
Explanation: The frequency of a is 4, b is 2 and c is 1 so there are 2 characters with even frequency.

Input: s = "zzaccz"

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed	Attempts : Correct / Total
1115 / 1115	**1 / 1**
	Accuracy : 100%

Points Scored Time Taken

Java (21) Start Timer

```java
class Solution {
    public int count(String s) {
        int[] freq = new int[26];
        for (int i = 0; i < s.length(); i++) {
            freq[s.charAt(i) - 'a']++;
        }

        int evenCount = 0;
        for (int count : freq) {
            if (count > 0 && count % 2 == 0) {
                evenCount++;
            }
        }
        return evenCount;
    }
}
```

Custom Input Compile & Run Submit

Problem Submissions Comments

Shortest substring containing all vowels

Difficulty: **Medium** Accuracy: **65.16%** Submissions: **1K+** Points: **4**

You are given two strings, **s1** and **s2**, where s1 contains **distinct lowercase vowels** (a, e, i, o, u), and s2 contains **lowercase English letters**. Your task is to find the **length** of the **shortest** contiguous **substring** within s2 that contains all the **vowels** present in s1 at least **once**, in any **order**.

Note: If there is no such substring in s2, return **-1**.

Examples:

Input: s1 = "ae", s2 = "acbaudeq"
Output: 4

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed
1120 / 1120

Attempts : Correct / Total
1 / 1
Accuracy : 100%

Points Scored

Time Taken

Java (21) Start Timer

```java
        int n = s2.length();
        int requiredCount = s1.length();
        int[] target = new int[256];
        boolean[] isRequired = new boolean[256];

        for (char c : s1.toCharArray()) {
            target[c]++;
            isRequired[c] = true;
        }

        int[] currentMap = new int[256];
        int left = 0, right = 0;
        int found = 0;
        int minLen = Integer.MAX_VALUE;

        while (right < n) {
            char rChar = s2.charAt(right);
            if (isRequired[rChar]) {
                if (currentMap[rChar] == 0) {
                    found++;
                }
                currentMap[rChar]++;
            }

            while (found == requiredCount) {
                minLen = Math.min(minLen, right - left + 1);
                char lChar = s2.charAt(left);
                if (isRequired[lChar]) {
                    currentMap[lChar]--;
                    if (currentMap[lChar] == 0) {
                        found--;
                    }
                }
                left++;
            }
            right++;
        }

        return (minLen == Integer.MAX_VALUE) ? -1 : minLen;
    }
}
```

Custom Input Compile & Run Submit